MODELO DE DOCUMENTAÇÃO TÉCNICA

Modelo contínuo para planejamento, desenvolvimento, implantação, infraestrutura e operação

Campo	Informação
Nome do projeto	[Preencher]
Versão atual do documento	0.1
Responsável técnico	[Preencher]
Equipe / turma	[Preencher]
Repositório principal	[Preencher]
Ambiente publicado	[Preencher / Não aplicável]
Última atualização	[dd/mm/aaaa]
Status	Em elaboração

Orientação de preenchimento: Este documento deve ser atualizado continuamente. Não crie um novo documento a cada etapa do projeto. Atualize as seções existentes, registre mudanças no histórico e marque como 'Planejado', 'Em andamento', 'Validado' ou 'Não aplicável' os itens que ainda não se aplicam.

Documento-base reutilizável para projetos de software com qualquer quantidade de etapas.

1. Controle do documento

Orientação de preenchimento: Mantenha este bloco sempre atualizado. Ele funciona como o ponto de entrada para saber o estado real do projeto e da documentação.

1.1 Identificação

Item	Valor
Projeto	[Preencher]
Produto / sistema	[Preencher]
Objetivo resumido	[Preencher em 1 ou 2 frases]
Responsável técnico	[Preencher]
Stakeholders principais	[Preencher]
Repositório	[Link]
Documentação complementar	[Links]

1.2 Histórico de versões

Versão	Data	Responsável	Alteração realizada	Status
0.1	[data]	[nome]	Criação inicial do documento	Rascunho

1.3 Estado das áreas documentais

Área	Estado	Última revisão	Responsável	Observações
Visão e escopo	Planejado			
Requisitos	Planejado			
Prototipação e UX	Planejado			
Modelagem e arquitetura	Planejado			
Código e versionamento	Planejado			
Testes e qualidade	Planejado			
Dados e banco de dados	Planejado			
Publicação e ambientes	Planejado			
Infraestrutura	Planejado			
Segurança	Planejado			
Operação, logs e backup	Planejado			
Continuidade / failover	Planejado			

Orientação de preenchimento: Sugestão de estados: Planejado, Em andamento, Parcialmente validado, Validado, Obsoleto, Não aplicável.

1.4 Registro de decisões técnicas

ID	Data	Decisão	Motivo	Alternativas consideradas	Impacto
ADR-001					
ADR-002					

Orientação de preenchimento: Registre decisões relevantes, por exemplo: escolha de framework, banco, arquitetura, distro Linux, estratégia de deploy, autenticação ou tecnologia de failover. Evite justificar decisões apenas verbalmente.

2. Visão geral do projeto

2.1 Contexto e problema

Orientação de preenchimento: Explique o problema que motivou o projeto, quem é afetado, em qual contexto ele ocorre e por que a solução é necessária.

[Descreva o contexto, a situação atual e o problema principal.]

2.2 Objetivo da solução

[Descreva tecnicamente o que o sistema deve entregar e qual resultado se espera alcançar.]

2.3 Escopo

Dentro do escopo	Fora do escopo
[Funcionalidade / responsabilidade]	[Item explicitamente não contemplado]

2.4 Público e usuários

Perfil	Necessidade	Como utiliza a solução	Nível de acesso
[Usuário]			
[Administrador]			

2.5 Critérios de sucesso

Critério	Métrica / evidência	Meta	Situação atual
[Ex.: executar fluxo principal]	[Teste / métrica]	[Meta]	[Status]

3. Requisitos

Orientação de preenchimento: Esta seção cresce com o projeto. Requisitos podem ser refinados e priorizados ao longo do desenvolvimento, mas devem manter rastreabilidade com testes e entregas.

3.1 Requisitos funcionais

ID	Requisito	Prioridade	Origem	Status	Evidência / teste
RF-001	[O sistema deve...]	Alta	[Stakeholder]	Planejado	
RF-002					

3.2 Requisitos não funcionais

ID	Categoria	Requisito	Critério mensurável	Status
RNF-001	Desempenho	[Preencher]	[Ex.: resposta < 2 s]	
RNF-002	Segurança	[Preencher]	[Critério]	
RNF-003	Usabilidade	[Preencher]	[Critério]	

3.3 Regras de negócio

ID	Regra	Origem	Impacto no sistema
RN-001	[Preencher]	[Stakeholder / documento]	[Preencher]

3.4 Restrições e premissas

Tipo	Descrição	Impacto
Restrição	[Ex.: linguagem Python obrigatória]	[Preencher]
Premissa	[Ex.: acesso à internet disponível]	[Preencher]

4. Processo de trabalho e gestão

4.1 Organização do trabalho

[Descreva método adotado: Scrum, Kanban, fluxo híbrido, papéis, cerimônias e frequência.]

4.2 Backlog e planejamento

Item	Descrição / link
Ferramenta de gestão	[GitHub Projects / Trello / ClickUp / etc.]
Backlog principal	[Link]
Critério de priorização	[Preencher]
Definition of Ready	[Preencher]
Definition of Done	[Preencher]

4.3 Entregas e marcos

Marco	Resultado esperado	Data prevista	Data real	Status
M1	[Preencher]			
M2	[Preencher]			
M3	[Preencher]			

Orientação de preenchimento: Os marcos podem coincidir com etapas acadêmicas ou comerciais, mas o documento não deve depender de uma quantidade fixa de fases.

5. Prototipação e experiência de uso

5.1 Fluxos principais

Fluxo	Ator	Entrada	Passos resumidos	Resultado esperado
FL-01	[Usuário]	[Entrada]	[Passos]	[Saída]

5.2 Protótipos

Tela / interface	Objetivo	Link / imagem	Status	Observações
[Tela inicial]	[Objetivo]	[Link]	Rascunho	

5.3 Padrões de interface

[Registre decisões de navegação, acessibilidade, layout, feedback ao usuário, responsividade e consistência visual.]

6. Modelagem e arquitetura

> **Orientação de preenchimento:** Documente a visão necessária para entender o sistema. Evite diagramas excessivamente detalhados que rapidamente fiquem desatualizados. Prefira visão de contexto, componentes principais, domínio e fluxos críticos.

6.1 Visão arquitetural

[Descreva o estilo arquitetural adotado: monólito, cliente-servidor, MVC, camadas, API, microsserviços etc.]

6.2 Diagrama de contexto

[Inserir diagrama ou link: sistema, usuários e sistemas externos.]

6.3 Componentes principais

Componente	Responsabilidade	Tecnologia	Dependências
[Interface]	[Responsabilidade]	[Framework]	[Dependências]
[Lógica de negócio]			
[Persistência]			

6.4 Modelagem orientada a objetos

Classe / abstração	Responsabilidade	Atributos principais	Métodos principais	Relacionamentos
[Classe]	[Responsabilidade]	[atributos]	[métodos]	[associação/herança/ etc.]

6.5 Diagramas UML e modelos

Artefato	Objetivo	Versão / data	Link / local
Caso de uso	[Preencher]		
Classe	[Preencher]		
Sequência	[Preencher]		
Outro	[Preencher]		

7. Stack, dependências e ambientes de desenvolvimento

7.1 Tecnologias

Categoria	Tecnologia	Versão principal	Motivo da escolha
Linguagem	Python	[x]	[Preencher]
Interface	[Tkinter / Flask / Django / Streamlit / etc.]		
Banco	[Preencher]		
Testes	[pytest / unittest / etc.]		

7.2 Dependências

Dependência	Finalidade	Arquivo de controle	Observação

[biblioteca]	[uso]	requirements.txt / pyproject.toml	

7.3 Preparação do ambiente

Etapa	Comando / ação	Resultado esperado
Clonar repositório	git clone ...	Código disponível localmente
Criar ambiente virtual	python -m venv .venv	Ambiente isolado criado
Instalar dependências	pip install -r requirements.txt	Dependências instaladas
Executar projeto	[comando]	[resultado]

Orientação de preenchimento: Nunca registre senhas, tokens, chaves privadas ou segredos diretamente neste documento. Registre apenas o nome das variáveis de ambiente e onde obter/configurar seus valores.

7.4 Variáveis de ambiente e configurações

Variável	Finalidade	Obrigatória?	Exemplo seguro
APP_ENV	Ambiente de execução	Sim	development
[VARIAVEL]	[Descrição]	[Sim/Não]	[exemplo sem segredo]

8. Código-fonte e versionamento

8.1 Repositório

Item	Valor
Plataforma	[GitHub / GitLab / outro]
URL	[Link]
Branch principal	[main / master]
Estratégia de branches	[Preencher]
Política de commits	[Preencher]

8.2 Estrutura do projeto

[Apresente somente as pastas e arquivos relevantes. Exemplo: src/, tests/, docs/, scripts/, requirements.txt.]

8.3 Fluxo de alteração

- ☐ Criar ou selecionar item do backlog.
- ☐ Criar branch ou trabalhar conforme estratégia definida.
- ☐ Implementar alteração mantendo padrões de código.
- ☐ Executar testes e validações.
- ☐ Registrar mudanças necessárias na documentação.
- ☐ Abrir Pull Request / Merge Request quando aplicável.
- ☐ Revisar e integrar à branch principal.

9. Testes, validação e qualidade

9.1 Estratégia de testes

Tipo de teste	Objetivo	Ferramenta	Quando executar
Unitário	[Preencher]	[pytest/unittest]	[local/CI]

Integração	[Preencher]	[ferramenta]	[momento]
Aceitação	[Preencher]	[manual/automatizado]	[momento]
Interface	[Preencher]	[ferramenta]	[momento]

9.2 Registro de testes

ID	Requisito relacionado	Cenário	Resultado esperado	Resultado obtido	Status	Evidência
T-001	RF-001	[Cenário]	[Esperado]	[Obtido]	Pendente	[print/log/link]

9.3 Defeitos e correções relevantes

ID	Problema	Causa	Correção	Teste de regressão	Status
BUG-001	[Descrição]	[Causa]	[Correção]	[Teste]	[Status]

10. Dados e banco de dados

10.1 Visão dos dados

[Explique quais dados o sistema armazena, sua origem, ciclo de vida e finalidade.]

10.2 Modelagem

Artefato	Descrição	Link / local	Última atualização
Modelo conceitual	[Preencher]		
Modelo lógico	[Preencher]		
Modelo físico	[Preencher]		

10.3 Estruturas principais

Tabela / coleção	Finalidade	Chave primária	Relacionamentos	Observações
[nome]	[Finalidade]	[PK]	[FK/relacionamentos]	

10.4 Migrações, carga e recuperação

[Descreva como criar o banco, aplicar migrações, popular dados de teste, exportar e restaurar dados.]

11. Publicação, ambientes e entrega

11.1 Ambientes

Ambiente	Finalidade	Endereço / host	Fonte da versão	Status
Desenvolvimento	[Uso]	[local]	[branch]	
Homologação	[Uso]	[URL/host]	[branch/tag]	
Produção	[Uso]	[URL/host]	[tag/release]	

11.2 Processo de publicação

Passo	Ação / comando	Validação
1	[Preparar build / dependências]	[Critério]
2	[Publicar]	[Critério]
3	[Executar smoke test]	[Critério]
4	[Registrar versão]	[Critério]

11.3 Rollback

[Descreva como retornar para a última versão estável em caso de falha.]

12. Infraestrutura e provisionamento

Orientação de preenchimento: Esta seção pode permanecer resumida ou marcada como 'Não aplicável' em projetos sem infraestrutura própria. Quando houver servidor local ou em nuvem, documente somente o necessário para reproduzir e operar o ambiente.

12.1 Inventário

Recurso	Tipo	Sistema / distribuição	Função	Endereço / identificação
Servidor 01	VM / físico	[Linux distro]	[Aplicação]	[IP/hostname]

12.2 Virtualização

Item	Configuração
Hipervisor / plataforma	[VirtualBox / Proxmox / VMware / cloud]
CPU	[Preencher]
Memória	[Preencher]
Disco	[Preencher]
Rede virtual	[Preencher]

12.3 Rede

Parâmetro	Valor / regra	Justificativa
Hostname	[Preencher]	
IP / DHCP	[Preencher]	
Gateway	[Preencher]	
DNS	[Preencher]	
Portas expostas	[Preencher]	
Segmentação	[Preencher]	

12.4 Usuários, grupos e permissões

Usuário / grupo	Finalidade	Permissões	Acesso permitido
[usuario_app]	[Serviço]	[mínimo necessário]	[pastas/serviços]

12.5 Dependências do servidor

Pacote / serviço	Finalidade	Instalação	Configuração principal
Python	[runtime]	[comando]	[versão]

Git	[deploy]	[comando]	
[serviço]			

13. Execução como serviço e automação

13.1 Processos da aplicação

Processo / serviço	Usuário	Porta	Comando	Dependências
[app.service]	[usuario]	[porta]	[comando]	[deps]

13.2 Systemd

[Inclua o nome da unit, objetivo, caminho do arquivo, usuário, WorkingDirectory, ExecStart, política de restart e comandos de gerenciamento.]

Ação	Comando
Recarregar units	sudo systemctl daemon-reload
Habilitar serviço	sudo systemctl enable <servico>
Iniciar	sudo systemctl start <servico>
Ver status	systemctl status <servico>
Ver logs	journalctl -u <servico>

13.3 Shell scripts e automações

Script	Objetivo	Entrada	Saída	Execução	Responsável
deploy.sh	[Deploy]	[parâmetros]	[resultado]	[manual/cron/systemd]	
backup.sh	[Backup]				

14. Segurança

14.1 Princípios adotados

- ☐ Princípio do menor privilégio para usuários e serviços.
- ☐ Segredos fora do código e da documentação.
- ☐ Atualizações de segurança aplicadas regularmente.
- ☐ Somente portas necessárias expostas.
- ☐ Logs suficientes para auditoria e diagnóstico.
- ☐ Backups testados e armazenados separadamente quando possível.

14.2 Firewall

Regra	Origem	Destino / porta	Ação	Justificativa
FW-001	[rede/origem]	[porta]	Permitir	[serviço necessário]

14.3 Acessos e autenticação

Recurso	Método	Quem acessa	Proteção aplicada
Servidor	SSH	[grupo]	[chaves / restrições]

Aplicação	[método]	[perfil]	[controle]
Banco	[método]	[perfil]	[controle]

15. Monitoramento, logs e diagnóstico

15.1 Logs

Origem	Local / comando	Conteúdo	Retenção
Aplicação	[arquivo / journalctl]	[erros/eventos]	[período]
Sistema	/var/log/...	[eventos]	[período]

15.2 Indicadores de saúde

Indicador	Como verificar	Condição saudável	Ação em caso de falha
Serviço ativo	systemctl status ...	active (running)	[ação]
Porta respondendo	ss / curl	[resposta]	[ação]
Disco	df -h	[limite]	[ação]
Memória	free -h	[limite]	[ação]

15.3 Procedimentos de diagnóstico

Sintoma	Verificações	Causa provável	Ação corretiva
Aplicação não abre	[processo, porta, logs]	[preencher]	[preencher]
Banco indisponível	[serviço, conexão, credenciais]	[preencher]	[preencher]

16. Backup, recuperação e continuidade

16.1 Política de backup

Item protegido	Tipo	Frequência	Destino	Retenção	Responsável
Banco	[full/incremental]	[frequência]	[destino]	[período]	
Arquivos da aplicação	[tipo]	[frequência]	[destino]	[período]	

16.2 Restauração

Cenário	Pré-requisitos	Procedimento	Tempo estimado	Último teste
Falha do banco	[backup disponível]	[passos]	[tempo]	[data]
Perda do servidor	[infra alternativa]	[passos]	[tempo]	[data]

16.3 Failover e contingência

[Descreva quando e como ocorre failover, qual componente secundário assume, como os dados permanecem consistentes e como retornar ao ambiente principal.]

Falha	Detecção	Ação automática/manual	Destino alternativo	Validação
Servidor principal indisponível	[método]	[ação]	[secundário]	[teste]

17. Atualização e deploy pelo Git

17.1 Estratégia de atualização

Etapa	Comando / ação	Validação
Verificar estado	git status	[sem alterações inesperadas]
Atualizar código	git pull / checkout tag	[versão correta]
Atualizar dependências	pip install -r requirements.txt	[sem erros]
Reiniciar serviço	systemctl restart <serviço>	[ativo]
Validar	[smoke test]	[resultado esperado]

17.2 Procedimento seguro de atualização

- ☐ Confirmar existência de backup recente.
- ☐ Registrar versão atual.
- ☐ Baixar versão alvo.
- ☐ Aplicar migrações quando necessário.
- ☐ Reiniciar serviços.
- ☐ Executar testes de saúde.
- ☐ Em caso de falha, executar rollback documentado.

18. Rastreabilidade

Orientação de preenchimento: Use esta matriz para conectar necessidade, implementação e evidência. Isso transforma a documentação em instrumento de validação, não apenas em texto descritivo.

Requisito	Artefato / classe	Commit / PR	Teste	Ambiente	Status
RF-001	[componente]	[hash/link]	T-001	[dev/prod]	[status]

19. Riscos, limitações e dívida técnica

19.1 Registro de riscos

ID	Risco	Probabilidade	Impacto	Mitigação	Responsável	Status
R-001	[Descrição]	[B/M/A]	[B/M/A]	[Ação]		

19.2 Limitações conhecidas

Limitação	Impacto	Solução temporária	Evolução futura
[Preencher]	[Impacto]	[Workaround]	[Plano]

19.3 Dívida técnica

Item	Motivo	Impacto	Prioridade	Ação futura
[Preencher]	[Motivo]	[Impacto]	[Prioridade]	[Ação]

20. Manutenção da documentação

20.1 Regras de atualização

- ☐ Atualizar a documentação junto com mudanças relevantes no projeto.
- ☐ Não duplicar informação que já possui fonte confiável no código ou ferramenta; preferir links.
- ☐ Manter diagramas em nível suficiente para explicar o sistema, evitando excesso de detalhe volátil.
- ☐ Registrar decisões técnicas importantes no momento em que forem tomadas.
- ☐ Vincular requisitos a testes e evidências.
- ☐ Revisar o documento antes de cada marco importante, publicação ou entrega.
- ☐ Marcar explicitamente itens que ainda não se aplicam ao projeto.

20.2 Checklist de revisão técnica

- ☐ O objetivo e o escopo continuam corretos?
- ☐ Os requisitos refletem o comportamento atual?
- ☐ Arquitetura e diagramas representam o sistema implementado?
- ☐ As instruções de instalação e execução funcionam em ambiente limpo?
- ☐ Os comandos de deploy e rollback foram testados?
- ☐ Os testes possuem evidências recentes?
- ☐ A modelagem do banco corresponde ao esquema atual?
- ☐ Rede, usuários, permissões e firewall refletem o servidor real?
- ☐ Logs e procedimentos de diagnóstico estão utilizáveis?
- ☐ Backup e restauração foram testados?
- ☐ Não existem senhas, tokens ou segredos no documento?

21. Evidências e anexos

ID	Tipo	Descrição	Local / link	Relacionado a
EV-001	Captura de tela	[Descrição]	[link]	[Teste / requisito]
EV-002	Log	[Descrição]	[link]	[Incidente / teste]
EV-003	Diagrama	[Descrição]	[link]	[Arquitetura]

21.1 Sugestão de organização de evidências

Estrutura sugerida no repositório:

```
docs/
  arquitetura/
  requisitos/
  diagramas/
  testes/
  infraestrutura/
  operacao/
  evidencias/
```

22. Como este framework evolui ao longo de qualquer projeto

Orientação de preenchimento: A evolução ocorre pela maturidade de cada seção, e não pela criação de documentos separados. Um projeto simples pode preencher apenas algumas áreas; um projeto mais complexo pode amadurecer todas.

Momento típico do projeto	Seções que normalmente amadurecem
Início / descoberta	Visão, escopo, requisitos, processo de trabalho, riscos iniciais
Desenho da solução	Prototipação, arquitetura, modelagem, stack
Construção	Código, versionamento, dependências, testes, rastreabilidade
Persistência / integração	Banco de dados, APIs, migrações, segurança
Publicação	Ambientes, deploy, rollback, monitoramento
Infraestrutura	Servidor, rede, usuários, permissões, systemd, firewall
Operação	Logs, diagnóstico, backup, recuperação, continuidade e failover
Evolução	Decisões técnicas, riscos, dívida técnica, histórico e evidências

23. Guia rápido para o responsável pelo documento

- ☐ Comece preenchendo apenas o que já é conhecido.
- ☐ Não invente detalhes futuros: marque como Planejado.
- ☐ Ao implementar algo, atualize a seção correspondente.
- ☐ Ao testar, adicione resultado e evidência.
- ☐ Ao alterar arquitetura ou infraestrutura, registre a decisão e o motivo.
- ☐ Antes de uma entrega, revise o checklist de manutenção.
- ☐ Use o mesmo documento até o encerramento do projeto.

24. Referências e boas práticas adotadas

Este framework foi estruturado a partir de práticas de documentação técnica contínua, priorizando documentação simples, útil, ligada ao código e mantida ao longo do ciclo de vida. Entre os princípios utilizados estão: **objetivo do projeto, arquitetura e stack, acesso e execução, alterações e testes, atualização, monitoramento e manutenção.**

Referência principal: Guia.dev — "Documentação Técnica". A referência recomenda evitar documentação burocrática, manter informações úteis e atualizadas, usar links para fontes mais detalhadas e evitar duplicar informações voláteis que já são melhor representadas pelo código.

25. Campo livre para observações do projeto